

Build a Chat Agent

Build a one-page app that answers from *your* context — and learn to handle an API key safely. No coding needed.

 **5-day course handout · Build 3.** The final course build. It calls a real AI model, so it needs an **LLM API key** — which is why it's **not** part of the weekend taster (that ends at Build 2). Course-only.

Days 1–2 gave you a live site and a second brain. Day 3 crosses from *a website* to *an app that does something*: a chat grounded in your own documents, calling an AI model behind the scenes — plus the secret-handling every real app needs.

What's inside

1. **What an app actually is** — frontend, backend, data flow
2. **Start your own project** — your folder, not the course folder
3. **Choose & load your context** — the docs your agent answers from
4. **Build the chat** — grounded, honest answers
5. **API keys & secrets** — the real lesson of the day
6. **Add the fit-assessment** — the feature that sells you
7. **Deploy** — and watch your costs
8. **Prompt reference** — every prompt in one place
9. **You did it** — the checklist

What you'll need

- The setup from Days 1–2 (the **Claude app**, Node, Git, GitHub, Vercel) — see [prep/prerequisites.md](#) if anything's missing.
- **Context to ground the agent** — the sample persona (provided), or your own CV / product notes (ideally your **Build-2 second brain**).
- **A free OpenRouter API key** (no card needed) — the chat calls a free AI model. Steps in [MODEL-SETUP.md](#).

1. What an app actually is

The un-mystical version. Every app has two halves and one flow.

- **Frontend** — the page you *see* and click.
- **Backend** — the bit doing the work *behind* it.
- **Where code runs**: some in your **browser**, some on a **server**.

Every app is the same shape:

you type → it processes → you get a result

An **AI feature** is just one kind of "processes": your app **calls a model** and uses the answer.

See where you're headed: [alanoliver.dev](#) is a live chat that answers from someone's real context. Watch it answer — and notice when it *won't*. That honesty is the point.

Before any code, sketch it on paper: what goes **in** (a question) → what **happens** (call the model with your context) → what comes **out** (a grounded answer). Three boxes, two arrows. That's your app.

2. Start your own project

Same golden rule as Days 1–2:

The course repo is a textbook, not your workbook. Build in **your own folder** that you own. (More: [prep/working-on-your-own-projects.md](#).)

In the **Code** tab, paste:

```
Make a new folder called "my-chat-agent" in my home directory – NOT inside the jungli course folder. Copy the chat-agent skeleton (including the sample-persona/ folder) into it, start a fresh git repo, and confirm we're working in my-chat-agent.
```

Full build flow lives in the starter's [PROMPTS.md](#).

3. Choose & load your context

Your chat answers from **documents you give it**. Same build, two flavours:

- **Career bot** (job-seeker) — your CV & projects, or your **Build-2 second brain** ([../second-brain](#)).
- **Product bot** (entrepreneur) — your offer, features, FAQ.

Today we use the sample persona, **Sam Rivera** ([sample-persona/persona.md](#)) — made-up and safe to share.

First, make sure the agent has actually read the material:

```
Read the files in sample-persona/. Summarise what you know about this person so I can check it's complete.
```

If the summary is right, the context is loaded.

Keep real data private. When you swap Sam for your own CV or product info, do it in your **private** project — never commit real personal data to a public repo.

4. Build the chat

Now the app itself:

```
Build a simple one-page web app with a chat box. When the user sends a message, call the LLM with the documents in sample-persona/ as context, and answer using ONLY that context. If the answer isn't there, say so.
```

Ask it about **Sam** — then try a question it *can't* know from the context.

The whole trick: the agent answers **only** from the context you gave it. If it isn't there, it says so — it never invents. Grounded answers you can trust.

5. API keys & secrets

This is the real lesson of the day.

- An **API** = asking another service (the AI model) to do something for you.
- An **API key** = your **password** to that service. Ours is a **free OpenRouter** key (no card), but it's tied to your account, so it still stays **secret**.

Get a free key first: [MODEL-SETUP.md](#) — a free OpenRouter account, an API key, and a `:free` model. Then wire it in safely:

Put the OpenRouter API key in an `OPENROUTER_API_KEY` variable in a `.env` file. Add `.env` to `.gitignore` so it is never committed. Read the key from the environment in code – never hardcode it.

`.env` holds the secret · `.gitignore` keeps it off GitHub.

⚠️ **The classic disaster — never commit a key.** A key pushed to a public repo is in Git history **forever**, and bots scan public repos for keys and **run up your bill**. If you ever leak one, **revoke it immediately** and make a new one. `.env` + `.gitignore`, every time — no exceptions.

6. Add the fit-assessment

The feature that makes it *sell you* (job-seeker version):

Add a box where a recruiter pastes a job description. Return a structured fit assessment – strengths, gaps, and an overall fit rating – based only on the CV context.

Entrepreneur version: instead of a fit read, a "tell me about your product and pricing" answer — same build, product context.

7. Deploy — and watch your costs

Publish it, keeping the secret out of your code:

Deploy to Vercel. Add the API key as an environment variable in the Vercel project settings, not in the code.

The key lives in Vercel's settings — never in the code you push.

Cost & limits: every chat message is a real **API call**. On OpenRouter's free tier that's **~50 requests/day** per account (failed tries count) — plenty to build today. Hit the wall? Wait for the reset, switch `:free` model, or add **\$10 of credits** for 1,000/day. It's separate from your Claude plan; details in [MODEL-SETUP.md](#).

8. Prompt reference

Every prompt from this guide, in one place. Copy and customise.

Start your own project

```
Make a new folder called "my-chat-agent" in my home directory –
NOT inside the jungli course folder. Copy the chat-agent skeleton
(including the sample-persona/ folder) into it, start a fresh git
repo, and confirm we're working in my-chat-agent.
```

Load the context

```
Read the files in sample-persona/. Summarise what you know about
this person so I can check it's complete.
```

Build the chat

```
Build a simple one-page web app with a chat box. When the user
sends a message, call the LLM with the documents in sample-
persona/ as context, and answer using ONLY that context. If the
answer isn't there, say so.
```

Wire the API key safely (free OpenRouter key — see `MODEL-SETUP.md`)

Put the OpenRouter API key in an `OPENROUTER_API_KEY` variable in a `.env` file. Add `.env` to `.gitignore` so it is never committed. Read the key from the environment in code – never hardcode it.

Add the fit-assessment

Add a box where a recruiter pastes a job description. Return a structured fit assessment – strengths, gaps, and an overall fit rating – based only on the CV context.

Deploy

Deploy to Vercel. Add the API key as an environment variable in the Vercel project settings, not in the code.

9. You did it

If you followed every step, you now have:

- ☐ A plain-English grasp of **frontend, backend, and data flow**
- ☐ A **chat grounded in your context** — honest, never guessing
- ☐ Safe handling of an **API key**: `.env`, `.gitignore`, Vercel settings
- ☐ A **fit-assessment** (or product Q&A) feature
- ☐ A **deployed app** you can point at your own career or product

You've gone from a live page (Build 1) to a memory (Build 2) to a real, deployed **app** (Build 3). Next: **your own project**.

Built for the Jungli AI Learning Residency · AI & Agents track.